

Supplementary Information

Grammars

Primitive	Description
<u>Booleans</u>	
$=$	Two elements or two codes are equal.
$\in$	An element is in a code.
$\& \mid \wedge \mid !$	Standard logical operators (and, or, xor, not).
$\exists \exists! \forall$	Quantifiers (exists, exists-unique, and for-all).
<u>Elements</u>	
$e_1 \dots e_k$	Digits (or colors) of a code.
$C[i]$	Index of a code.
<u>Code</u>	
$[e_1 \dots e_n]$	An array of n digits (or colors).

Supplementary Table 1: The set of operations for representing the meaning of feedback in Mastermind.

Primitive	Description
<u>Booleans</u>	
$=$	Two elements or two sets are equal.
$\in$	An element is in a set.
$\& \mid \wedge \mid !$	Standard logical operators (and, or, xor, not).
$\exists \exists! \forall$	Quantifiers (exists, exists-unique, and for-all).
<u>Elements</u>	
$e_1 \dots e_k$	Integers from $1 \dots k$ corresponding to positions.
<u>Set Operations</u>	
$\{e\}$	A singleton set of element (position) e .
$\text{Union}(S, \{e\})$	Add element e to a set S .
$\text{Range}(i, j)$	Range of positions from i to j .

Supplementary Table 2: The set of operations for representing information about ButtonSet.

Table 1 shows the grammar used for interpreting feedback in the game of Mastermind and Table 2 shows the same for ButtonSet. The grammars are quite similar one that best-captured human logical inferences in several rule-based concept learning tasks [1]. The grammars allows for Boolean logic and first-order quantification to express propositional statements about the elements in the code and their positions. Grammars are shown above for both games.

Experiment 3 (Free Game-play)

Analyses

A question we did not address in the main text is the degree to which participants’ inefficiency stems from not processing the data fully versus purposefully picking low-information queries. These are, of course, not mutually exclusive possibilities and our model in fact predicts that they go hand-in-hand. If participants were fully processing the available information but lazily picking queries until the solution was determined, we would expect that they should get the correct answer after there was only one possible code/set. After removing redundant guesses (2% in both games), participants took on average 3.3 additional queries in Mastermind and 1.8 additional queries in ButtonSet after the solution was completely determined by available information.

In both games, many participants converged on low-complexity, low-effort strategies to solve the games — and increasingly so as they gained experience. In Mastermind, many participants learned a strategy of first guessing AAAA, BBBB, CCCC as their first three queries, which provides easily interpretable information at a low cost, i.e. revealing how many of each element is in the code. The proportion of players who used this strategy increased over time, from 11% on the first trial to 51% on the last ($B = 0.19$, $z = 8.0$, $p < 0.001$). This pattern was mirrored in ButtonSet, where participants increasingly favored simpler, more localized queries over time. On the first trial, the average guess had 4.7 buttons, but by the final trial the average guess had 3.2 buttons ($B = -0.14$, $t = -9.8$, $p < 0.001$).

Model Comparison

The results of Experiment 3 showed that people do not seem to be globally information-maximizing, but a key question is whether their performance was consistent with information-

maximizing subject to representational limits. We compared three nested models in Experiment 3: an “unbounded” model, which assumes only noise in the decision-process; a “leaky-posterior” model which additionally assumes noise in inference; and the “bounded” model, which uses program-approximated likelihood. The value of queries at any point in the game is given by,

$$U(\text{query}) \propto \sum_{h \in H} P(h) \sum_{o \in O} P(o \mid \text{query}, h) D_{KL} [P^*(H \mid o, \alpha, \beta) \parallel P(H)]. \quad (1)$$

The probability of choosing a given query is then given by a softmax with temperature τ . This determines the extent to which participants maximize the utility of a given query q given their beliefs,

$$P(\text{choose } q) \propto \exp\left(\frac{U(q)}{\tau}\right). \quad (2)$$

The leaky-posterior model assumes that after receiving information in each game, participants update their beliefs about which codes or sets are consistent with a Bernoulli likelihood with ϵ noise, i.e.,

$$P(E \mid h) = \begin{cases} 1 - \epsilon, & \text{if } h \text{ is consistent with } E. \\ \epsilon, & \text{otherwise.} \end{cases} \quad (3)$$

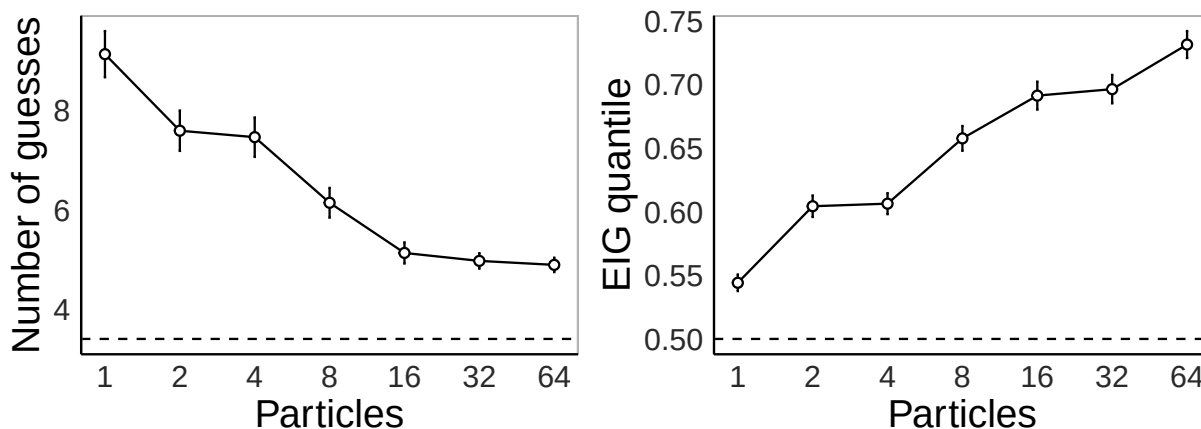
The bounded-representational model is an extension of this model, with the addition of the cost parameter α . The parameter for how well people optimize their approximations in the bounded-representation model, β , was set to 1 (not fit). We found maximum-likelihood parameters for each participant under each model using a greedy, stochastic search. For λ , τ , and α , candidates were iteratively proposed around the current best-fitting parameters using a log-normal, $\exp(\mathcal{N}(\log(\cdot), 0.1))$. For ϵ , candidates were proposed using a logistic, $\text{logit}^{-1}(\mathcal{N}(\text{logit}(\cdot), 0.1))$. Search was terminated when the log likelihood increased by less than 1 after 100 proposals.

Participants in both games were overall best fit by the bounded-representation model and only the bounded-representation model could account for participants’ systematically uninformative queries. In Mastermind, the mean ΔAIC favored the bounded representation model by 82.5 over the unbounded model and 72.3 over the leaky-posterior model; 37/39 participants had ΔAIC favoring the bounded representation model over the other two. In

ButtonSet, the mean ΔAIC favored the bounded representation model by 174.1 over the unbounded model and 125.4 over the leaky-posterior model; 37/42 participants had ΔAIC favoring the bounded representation model over both of the other two.

Participants’ queries in both games were less informative than chance-level under the unbounded model. Counting only the cases where there was information left to be gained, participants’ median query was the 25% most informative out of all possible queries in Mastermind and the 30% most informative in ButtonSet. They were at approximately chance-level informativeness in Mastermind under the leaky-posterior model, with the median query being 49% most informative; similarly, participants’ median query was the 40% most informative in ButtonSet under the leaky-posterior model. However, when fit to the bounded-representation model, participants’ queries were significantly more informative than a random query. The median query was the 70% most informative for Mastermind and 89% most informative for ButtonSet. As can be seen in the righthand panels of Figure 6 (main text), the modal EIG of participants’ queries was close to the (subjectively) information-maximizing query under this model.

Particle Filter



Supplementary Figure 1: Results of a particle filter model of Mastermind, showing the expected number of guesses to reach a solution (left) and the average informativity of each guess relative to other possible guesses (right) as a function of the number of particles (x-axis).

A key claim is that participants’ performance in these games cannot be explained by

standard sampling-based accounts of approximate inference and decision making. Intuitively, the worst these models can perform is if they take zero samples or track zero different hypotheses, and increasing the number of samples will only improve performance. To confirm this intuition, we ran a particle filter model on both games. At the start, the particle filter samples a fixed number of particles from the space of all possible codes or sets. After each guess, if a particle is consistent with the current feedback, the particle’s weight remains the same; otherwise, the weight is set to zero and the weights are then normalized so that they sum to 1. If all weights are zero (indicating that none of the particles are consistent with the feedback), the particle filter enters a “rejuvenation” step, where particles are resampled from the set of codes consistent with the most recent guess and feedback. In both games, even a particle filter with a single particle made queries that were more informative than chance. Figure 1 shows results for Mastermind. As the number of particles increases, the number of guesses to solve the game decreases (left) and the expected information gain increases (right).